



IBM Developer
SKILLS NETWORK

Winning Space Race with Data Science

Ali Abdullah
07-10-2026



Outline

- Executive Summary
- Introduction
- Methodology
- Results
- Conclusion
- Appendix

Executive Summary

- Developed a **predictive model** to determine **the success of SpaceX Falcon 9** first-stage landings.
- Collected launch data using the **SpaceX REST API** and **Wikipedia** web scraping. Performed data wrangling to clean and integrate multiple datasets.
- Conducted **exploratory data analysis** using Python visualizations and **SQL queries**.
- Built interactive maps using **Folium** and dashboards using **Plotly Dash**.
- Compared **multiple machine learning models** to identify **the best classifier** for launch success prediction.

Introduction

- Project background and context

SpaceX significantly **reduces launch costs** by recovering and reusing **the Falcon 9** first stage. Predicting whether a booster will land successfully helps estimate launch costs and mission reliability.

- Problems you want to find answers
 - Collect and integrate SpaceX launch data.
 - Explore relationships between launch variables and mission outcomes.
 - Visualize launch trends using charts and interactive maps.
 - Develop machine learning models to predict landing success. Compare model performance and identify the most accurate classifier.

Section 1

Methodology

Methodology

- **Content**
 - **Data Collection**
 - Retrieved launch records from the SpaceX REST API.
 - Collected launch site information through Wikipedia web scraping.
 - **Data Wrangling**
 - Cleaned missing and duplicate values.
 - Converted data types and engineered new features.
 - Combined datasets into a single analysis-ready dataset.
 - **Exploratory Data Analysis (EDA)**
 - Performed statistical analysis and data visualization using Python.
 - Executed SQL queries to explore launch trends and mission outcomes.
 - **Interactive Visual Analytics**
 - Built interactive launch site maps using Folium.
 - Developed an interactive dashboard using Plotly Dash.
 - **Predictive Analysis**
 - Trained and evaluated multiple classification models:
 - Logistic Regression
 - Support Vector Machine (SVM)
 - Decision Tree
 - K-Nearest Neighbors (KNN)
 - Compared model accuracy and selected the best-performing classifier.

Data Collection

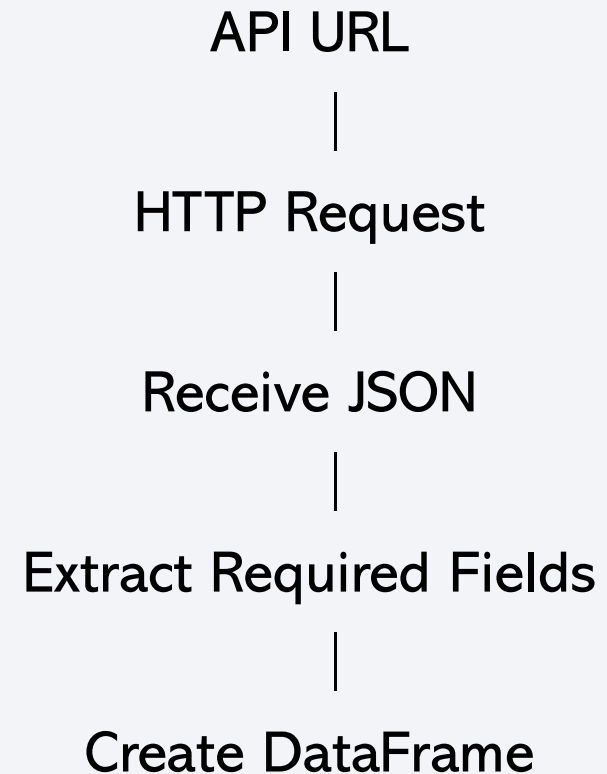
- Data Sources
 - SpaceX REST API
 - Wikipedia Launch Records
- Ways of Collecting Data
 - Extract launch data from SpaceX API
 - Scrape launch site information from Wikipedia
 - Merge both datasets
 - Save as CSV for further analysis

Data Collection – SpaceX API

- **Content**

- **Steps**

- Connected to SpaceX REST API
- Retrieved launch information
- Extracted:
 - Flight Number
 - Booster Version
 - Launch Site
 - Payload
 - Orbit
 - Landing Outcome
- Converted JSON into Pandas DataFrame



Data Collection - Scraping

- **Content**
- Download Wikipedia launch page
- Parse HTML using BeautifulSoup
- Extract launch tables
- Clean unwanted symbols
- Convert into DataFrame

Place your flowchart Wikipedia Page

Download HTML

BeautifulSoup

Locate Launch Tables

Extract Data

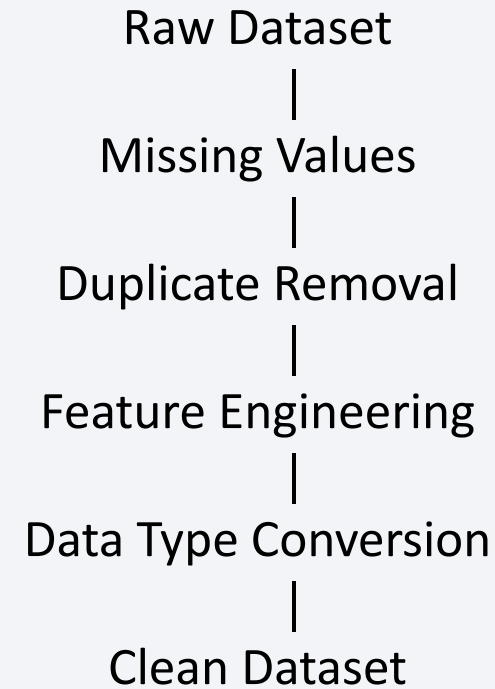
Clean Data

DataFrame web scraping here

Data Wrangling

- **Content**

- Removed missing values
- Deleted duplicates
- Converted date formats
- Converted payload to numeric
- Created landing success variable
- Merged datasets



EDA with Data Visualization

- **Content**
- Charts Created
 - Flight Number vs Launch Site
 - Payload vs Launch Site
 - Success Rate by Orbit
 - Payload vs Orbit
 - Yearly Launch Success
- Purpose
 - Understand trends
 - Identify important variables
 - Discover patterns before modeling

EDA with SQL

- **Content**
- Performed SQL queries to:
- List launch sites
- Calculate payload totals
- Find successful landings
- Identify booster performance
- Compare launch outcomes
- Rank landing outcomes

Build an Interactive Map with Folium

- **Content**

- Objects Added
- Launch site markers
- Success/Failure colored markers
- Railway lines
- Highways
- Coastline
- Distance circles

- **Purpose**

- Analyze launch site locations
- Measure proximity to infrastructure
- Understand geographical impact

Build a Dashboard with Plotly Dash

- **Content**

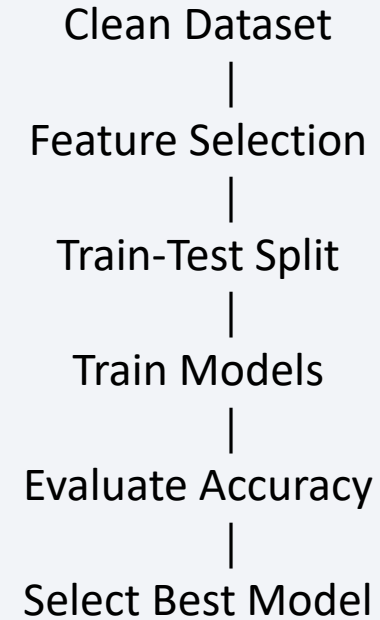
- Dashboard Includes
 - Launch Success Pie Chart
 - Payload Range Slider
 - Scatter Plot
 - Launch Site Filter

- **Purpose**

- Interactive exploration
- Dynamic filtering
- Compare launch sites

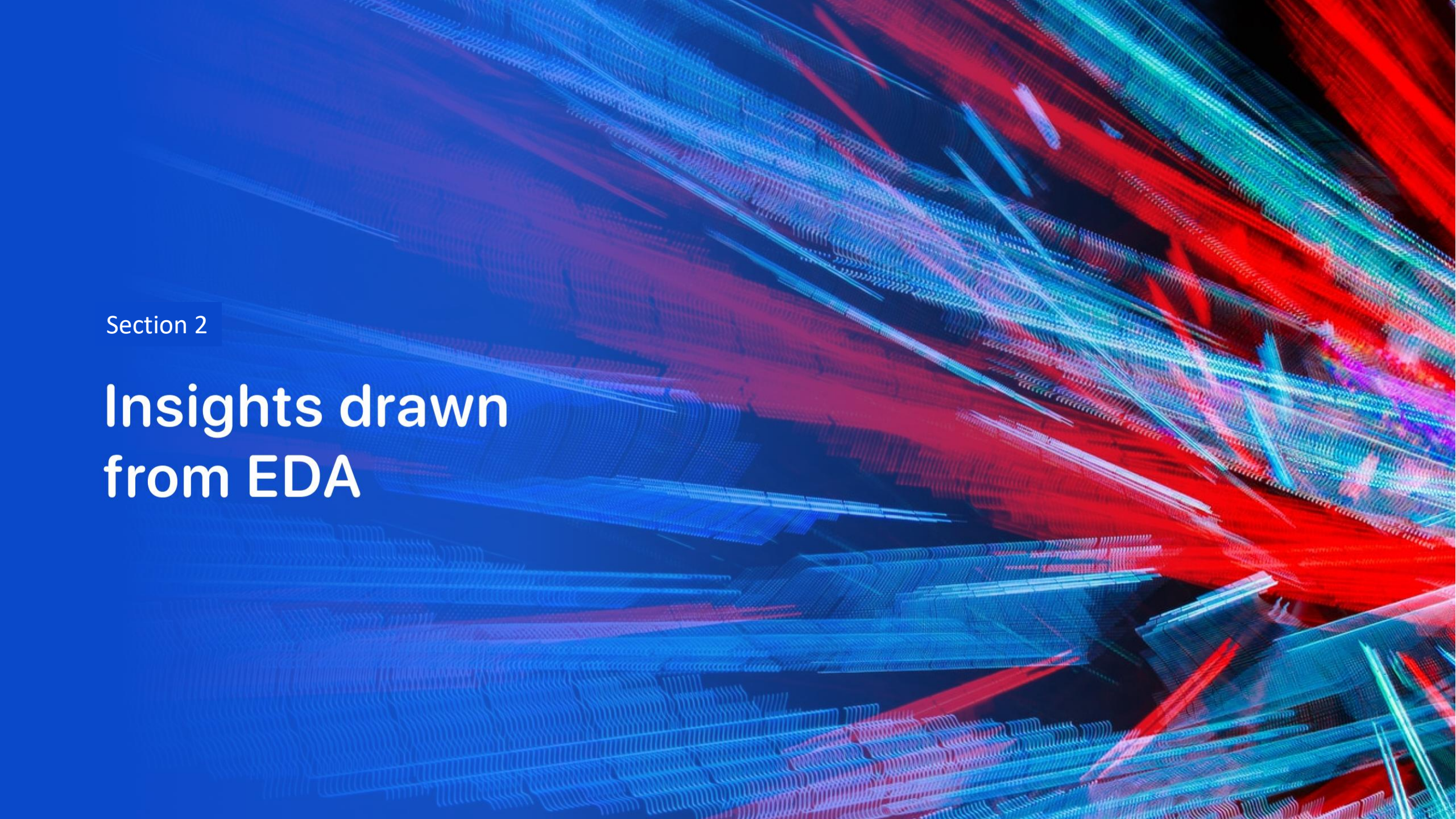
Predictive Analysis (Classification)

- **Content**
- **Models Built**
 - Logistic Regression
 - Support Vector Machine
 - Decision Tree
 - K-Nearest Neighbors
- **Evaluation**
 - Train/Test Split
 - Cross Validation
 - Hyperparameter Tuning
 - Accuracy Comparison



Results

- Exploratory data analysis results
- Interactive analytics demo in screenshots
- Predictive analysis results

The background of the slide is an abstract composition. It features a dark blue field on the left side, which transitions into a complex pattern of diagonal streaks in shades of blue, red, and cyan on the right. These streaks have a textured, almost woven appearance. Overlaid on this pattern is a faint, light blue grid that recedes into the distance, creating a sense of depth and perspective.

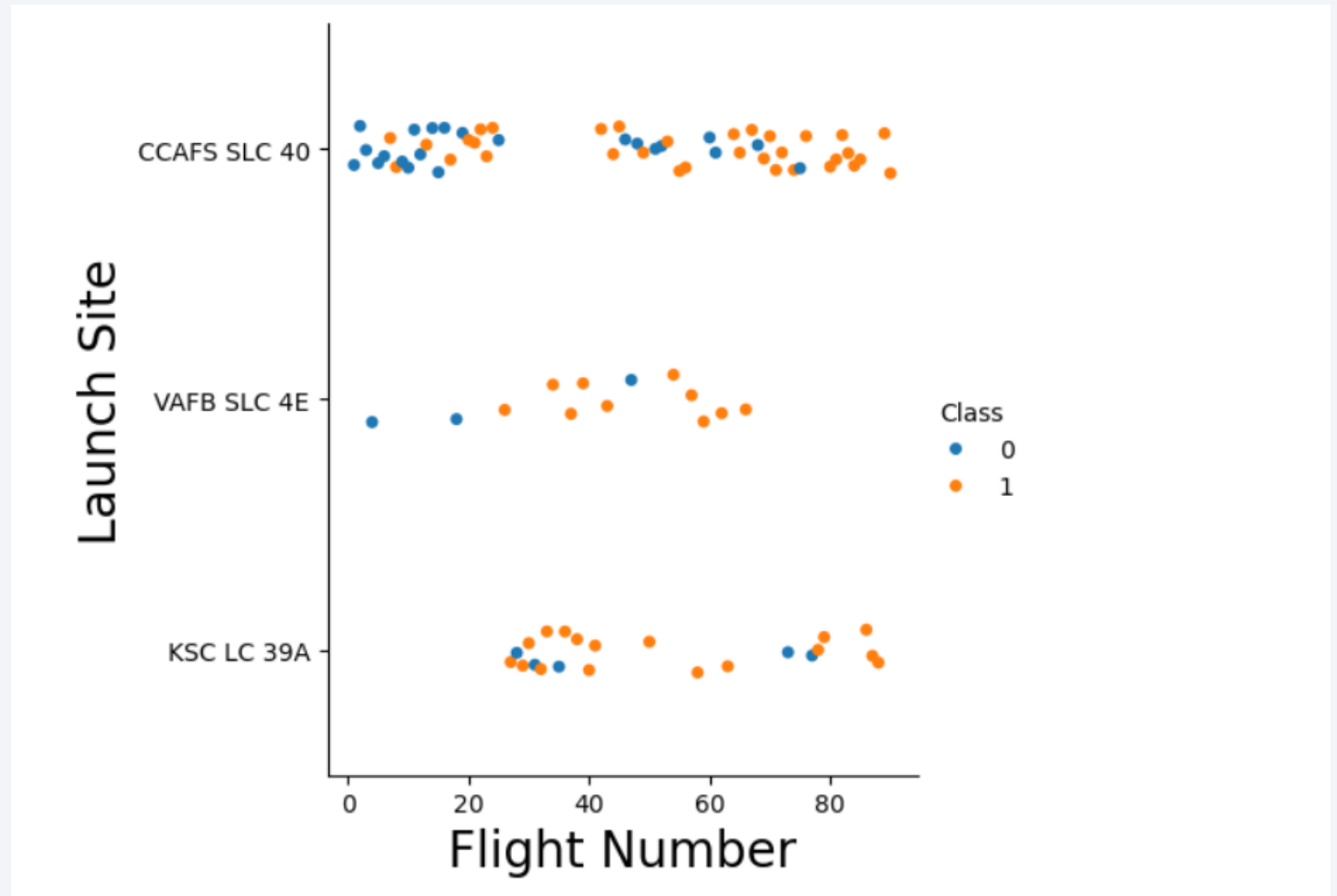
Section 2

Insights drawn from EDA

Flight Number vs. Launch Site

Observation

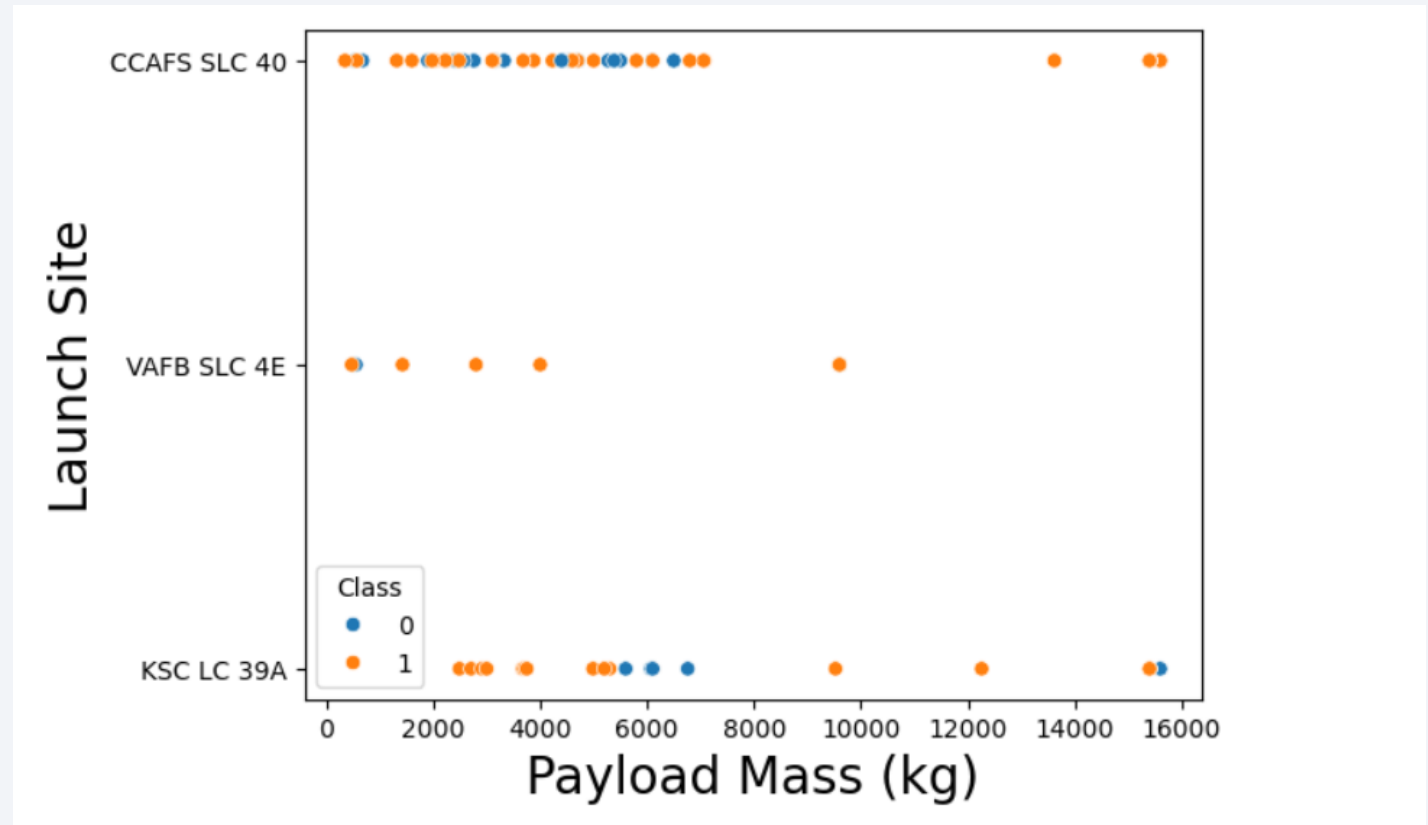
- **Success rate** increased with **flight number**.
- **Later launches** achieved **higher landing success**.
- **Kennedy Space Center** had the **highest success rate**.



Payload vs. Launch Site

Observation

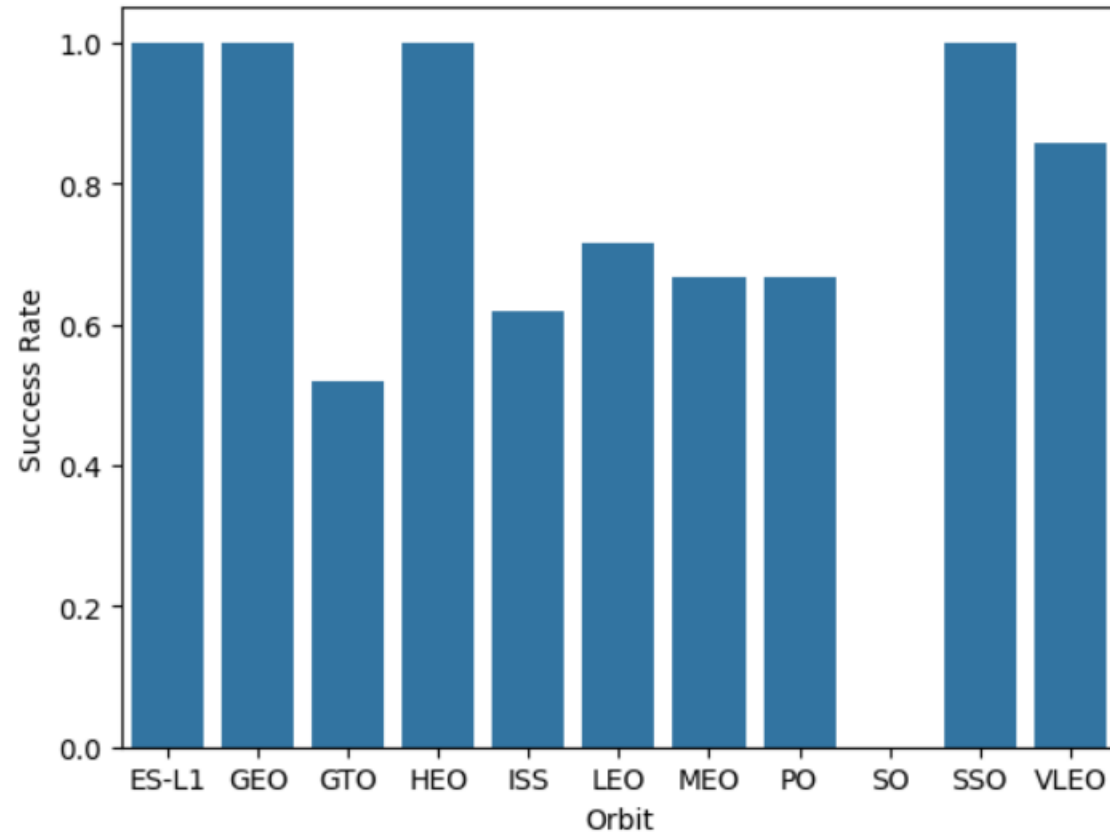
- **VAFB SLC 4E** launch site **have no rocket** launched for heavy payload mass (greater than 10000).



Success Rate vs. Orbit Type

Observation

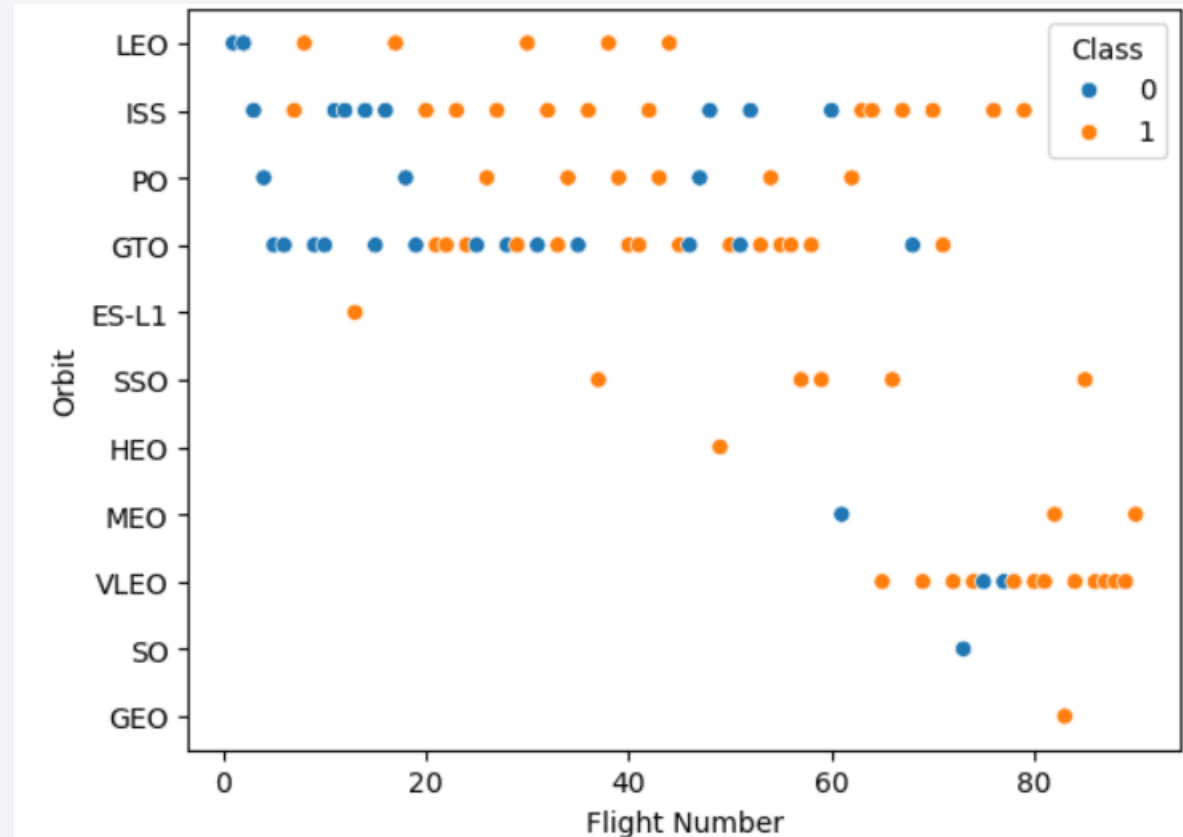
- **ES-L1, GEO, HEO** and **SSO** have **highest success rate** approximately above **95%**.
- **SO** has the **least success rate** below **10%**



Flight Number vs. Orbit Type

Observation

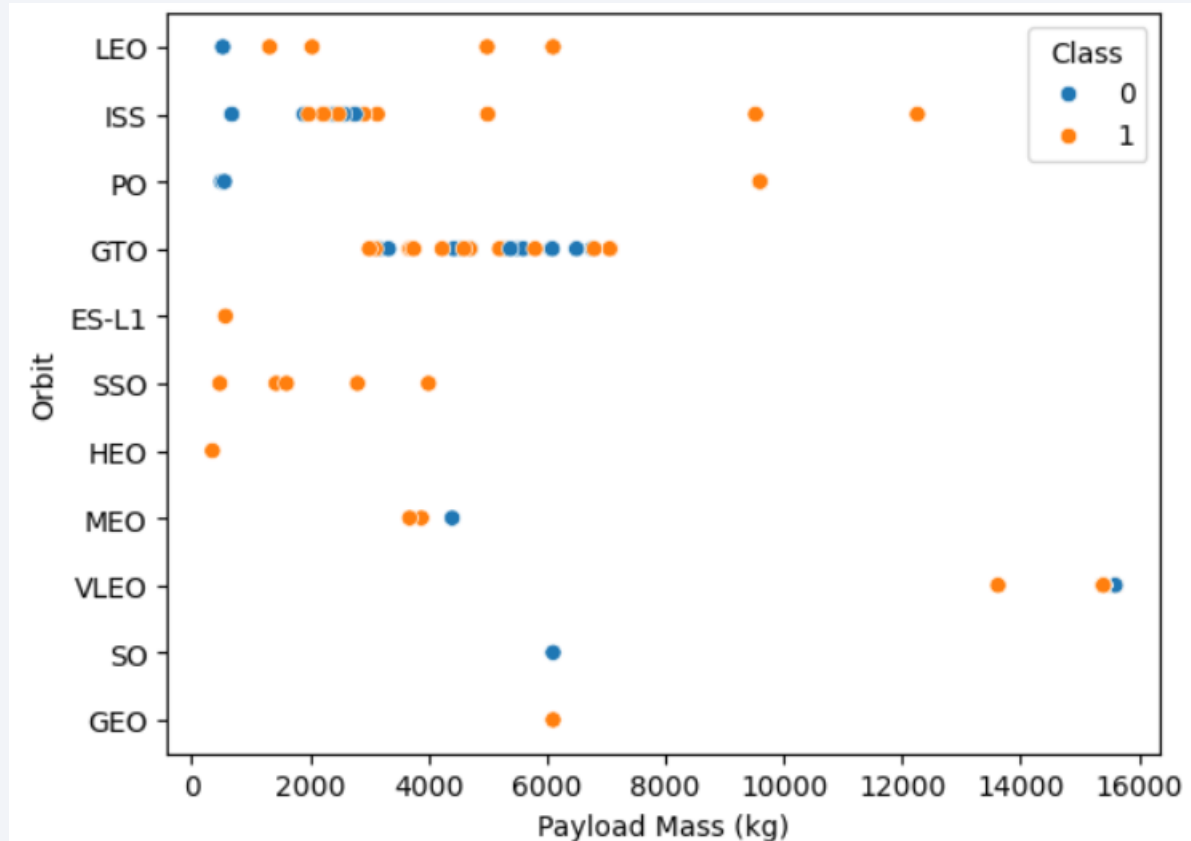
- You can observe that in the **LEO orbit**, success seems to be related to the number of flights.
- Conversely, in the **GTO orbit**, there appears to be no relationship between flight number and success.



Payload vs. Orbit Type

Observation

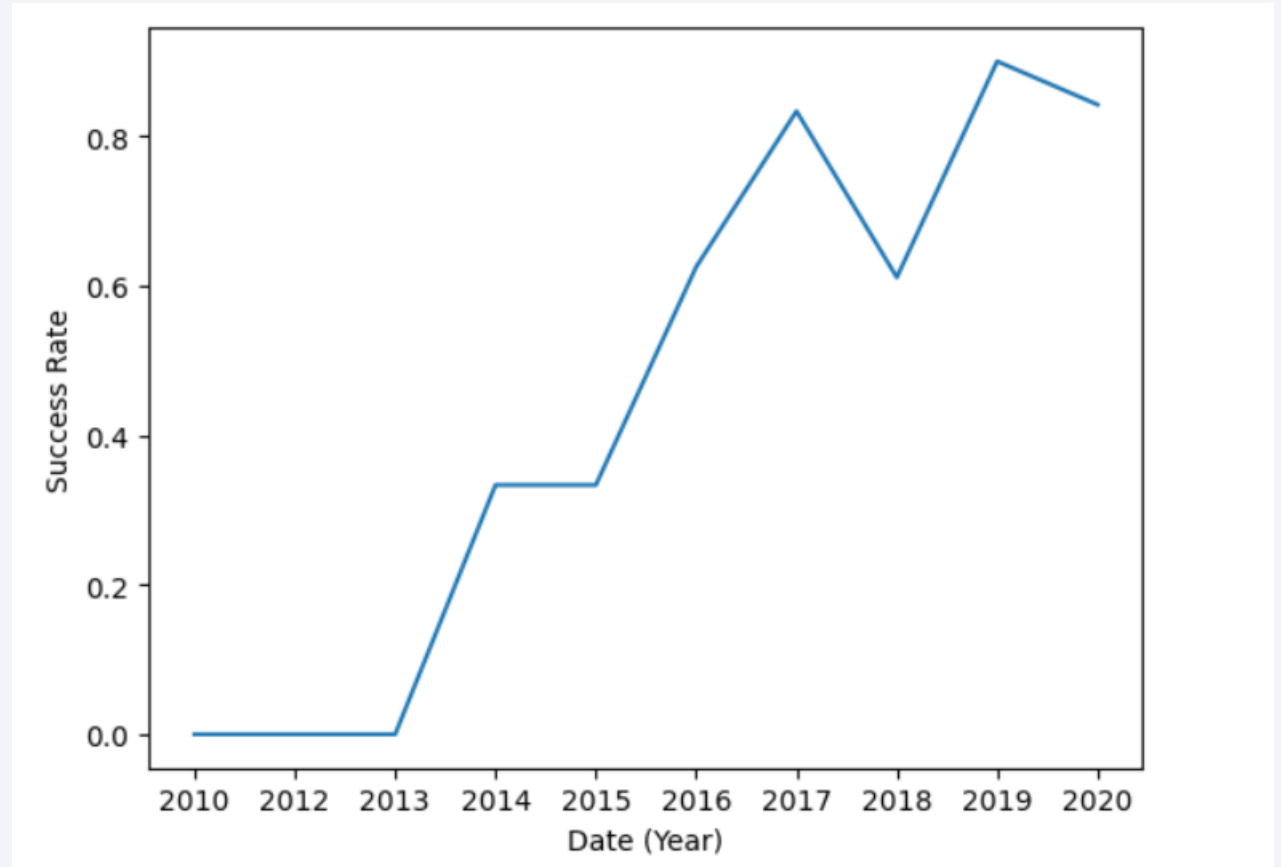
- With heavy payloads the **successful landing** or positive landing rate are more for **Polar, LEO** and **ISS**.
- However, for **GTO**, it's difficult to distinguish between successful and unsuccessful landings as both outcomes are present.



Launch Success Yearly Trend

Observation

- you can observe that **the success rate** since **2013** kept increasing till **2020**.



All Launch Site Names

Query

Select Distinct Launch_Site From SPACEXTBL;

Launch Sites

- CCAFS LC-40
- VAFB SLC-4E
- KSC LC-39A
- CCAFS SLC-40

Launch Site Names Begin with 'CCA'

Query

Select * From SPACEXTBL WHERE Launch_Site Like "CCA%";

Date	Time (UTC)	Booster_Version	Launch_Site	Payload	PAYLOAD_MASS_KG_	Orbit	Customer	Mission_Outcome	Landing_Outcome
2010-06-04	18:45:00	F9 v1.0 B0003	CCAFS LC-40	Dragon Spacecraft Qualification Unit	0	LEO	SpaceX	Success	Failure (parachute)
2010-12-08	15:43:00	F9 v1.0 B0004	CCAFS LC-40	Dragon demo flight C1, two CubeSats, barrel of Brouere cheese	0	LEO (ISS)	NASA (COTS) NRO	Success	Failure (parachute)
2012-05-22	7:44:00	F9 v1.0 B0005	CCAFS LC-40	Dragon demo flight C2	525	LEO (ISS)	NASA (COTS)	Success	No attempt
2012-10-08	0:35:00	F9 v1.0 B0006	CCAFS LC-40	SpaceX CRS-1	500	LEO (ISS)	NASA (CRS)	Success	No attempt
2013-03-01	15:10:00	F9 v1.0 B0007	CCAFS LC-40	SpaceX CRS-2	677	LEO (ISS)	NASA (CRS)	Success	No attempt
2013-12-03	22:41:00	F9 v1.1	CCAFS LC-40	SES-8	3170	GTO	SES	Success	No attempt
2014-01-06	22:06:00	F9 v1.1	CCAFS LC-40	Thaicom 6	3325	GTO	Thaicom	Success	No attempt
2014-04-18	19:25:00	F9 v1.1	CCAFS LC-40	SpaceX CRS-3	2296	LEO (ISS)	NASA (CRS)	Success	Controlled (ocean)
2014-07-14	15:15:00	F9 v1.1	CCAFS LC-40	OG2 Mission 1 6 Orbcomm-OG2 satellites	1316	LEO	Orbcomm	Success	Controlled (ocean)
2014-08-05	8:00:00	F9 v1.1	CCAFS LC-40	AsiaSat 8	4535	GTO	AsiaSat	Success	No attempt

Total Payload Mass

Query

```
SELECT BOOSTER_VERSION, SUM(PAYLOAD_MASS__KG_) FROM SPACEXTBL  
GROUP BY BOOSTER_VERSION;
```

Booster Version	Total_Payload
F9 B5 B1060.3	15600
F9 B5 B1060.2	15600
F9 B5 B1058.3	15600
F9 B5 B1056.4	15600
F9 B5 B1051.6	15600
F9 B5 B1051.4	15600
F9 B5 B1051.3	15600
F9 B5 B1049.7	15600
F9 B5 B1049.5	15600
F9 B5 B1049.4	15600

Average Payload Mass by F9 v1.1

Query

```
SELECT BOOSTER_VERSION, AVG(PAYLOAD_MASS__KG_) FROM SPACEXTBL  
WHERE BOOSTER_VERSION = 'F9 v1.1' GROUP BY BOOSTER_VERSION;
```

Booster Versions	AVG(PAYLOAD_MASS__KG_)
F9 v1.1	2928.4

First Successful Ground Landing Date

Query

- `SELECT * FROM SPACEXTBL WHERE LANDING_OUTCOME = 'Success (ground pad)' Order by Date Asc Limit 1;`

First Successful Ground Landing Date

- 2015-12-22

Successful Drone Ship Landing with Payload between 4000 and 6000

Query

- `SELECT BOOSTER_VERSION FROM SPACEXTBL WHERE LANDING_OUTCOME = 'Success (drone ship)' AND (PAYLOAD_MASS__KG_ > 4000 AND PAYLOAD_MASS__KG_ < 6000);`

Booster Versions

F9 FT B1022

F9 FT B1026

F9 FT B1021.2

F9 FT B1031.2

Total Number of Successful and Failure Mission Outcomes

Query

- `SELECT MISSION_OUTCOME, COUNT(*) FROM SPACEXTBL GROUP BY MISSION_OUTCOME;`

Mission Outcome	Total Number
Failure (in flight)	1
Success	98
Success	1
Success (payload status unclear)	1

Boosters Carried Maximum Payload

Query

- `SELECT BOOSTER_VERSION FROM SPACEXTBL WHERE PAYLOAD_MASS__KG_ = (SELECT MAX(PAYLOAD_MASS__KG_) FROM SPACEXTBL);`

Booster Version

F9 B5 B1048.4

F9 B5 B1049.4

F9 B5 B1051.3

F9 B5 B1056.4

F9 B5 B1048.5

F9 B5 B1051.4

F9 B5 B1049.5

F9 B5 B1060.2

F9 B5 B1058.3

F9 B5 B1051.6

F9 B5 B1060.3

F9 B5 B1049.7

2015 Launch Records

Query

- `SELECT SUBSTR(DATE,6,2) AS MONTH, LANDING_OUTCOME, BOOSTER_VERSION, LAUNCH_SITE FROM SPACEXTBL WHERE SUBSTR(DATE,0,5) = '2015' AND LANDING_OUTCOME = 'Failure (drone ship)';`

MONTH	Landing_Outcome	Booster_Version	Launch_Site
01	Failure (drone ship)	F9 v1.1 B1012	CCAFS LC-40
04	Failure (drone ship)	F9 v1.1 B1015	CCAFS LC-40

Rank Landing Outcomes Between 2010-06-04 and 2017-03-20

Query

- `SELECT LANDING_OUTCOME,COUNT(*) AS TOTAL_OUTCOME,RANK()
OVER(ORDER BY Count(*) DESC) AS RANKING FROM SPACEXTBL WHERE
DATE BETWEEN '2010-06-04' AND '2017-03-20' GROUP BY
LANDING_OUTCOME;`

Landing_Outcome	TOTAL_OUTCOME	RANKING
No attempt	10	1
Success (drone ship)	5	2
Failure (drone ship)	5	2
Success (ground pad)	3	4
Controlled (ocean)	3	4
Uncontrolled (ocean)	2	6
Failure (parachute)	2	6
Precluded (drone ship)	1	8

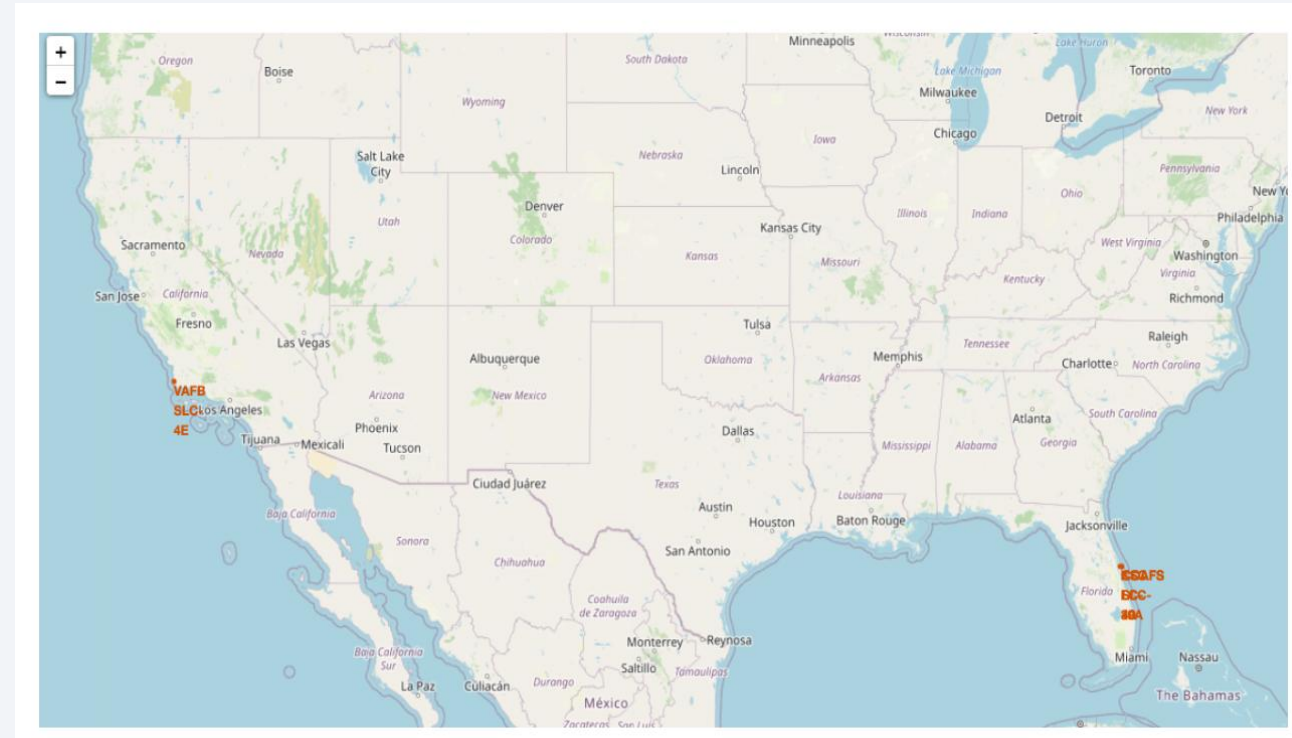
A satellite view of Earth from space, showing the curvature of the planet and city lights at night. The background is a deep blue gradient.

Section 3

Launch Sites Proximities Analysis

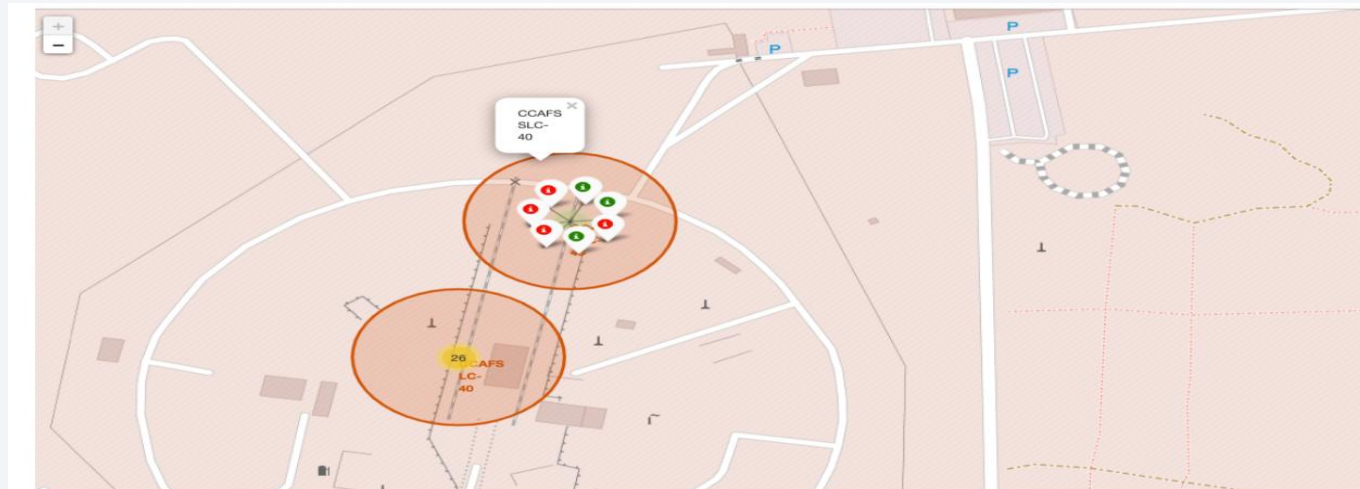
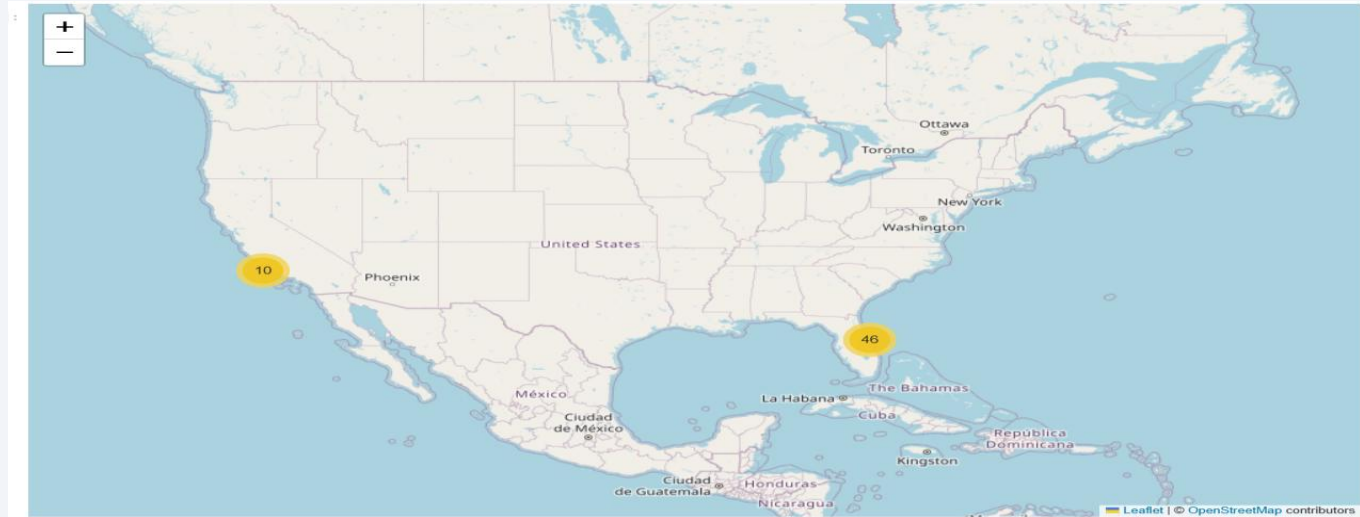
All Launch Sites on Map

- All **launch sites** displayed on world map.
- Launches concentrated in **Florida** and **California**.



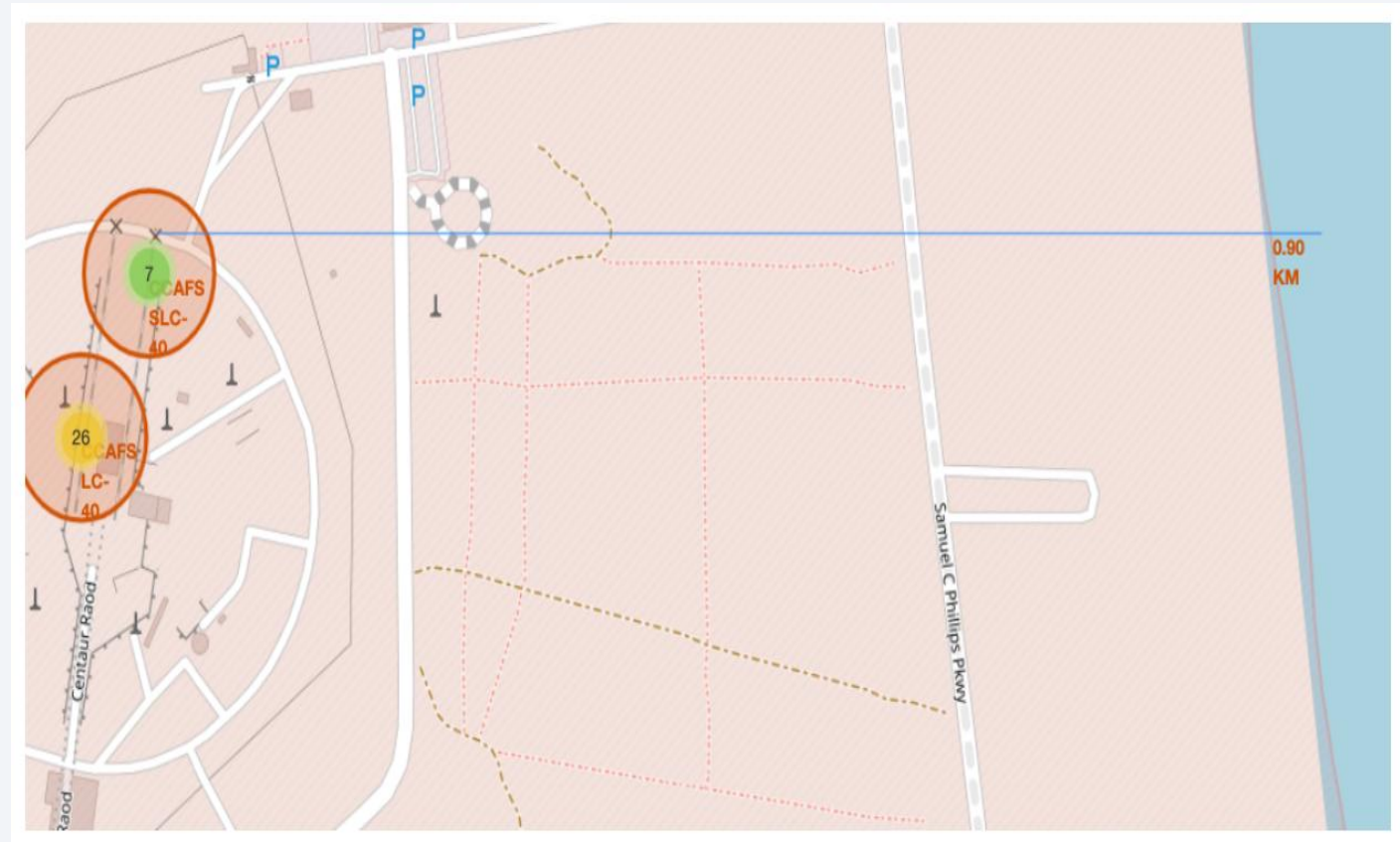
Marked Successful & Unsuccessful Launch Sites

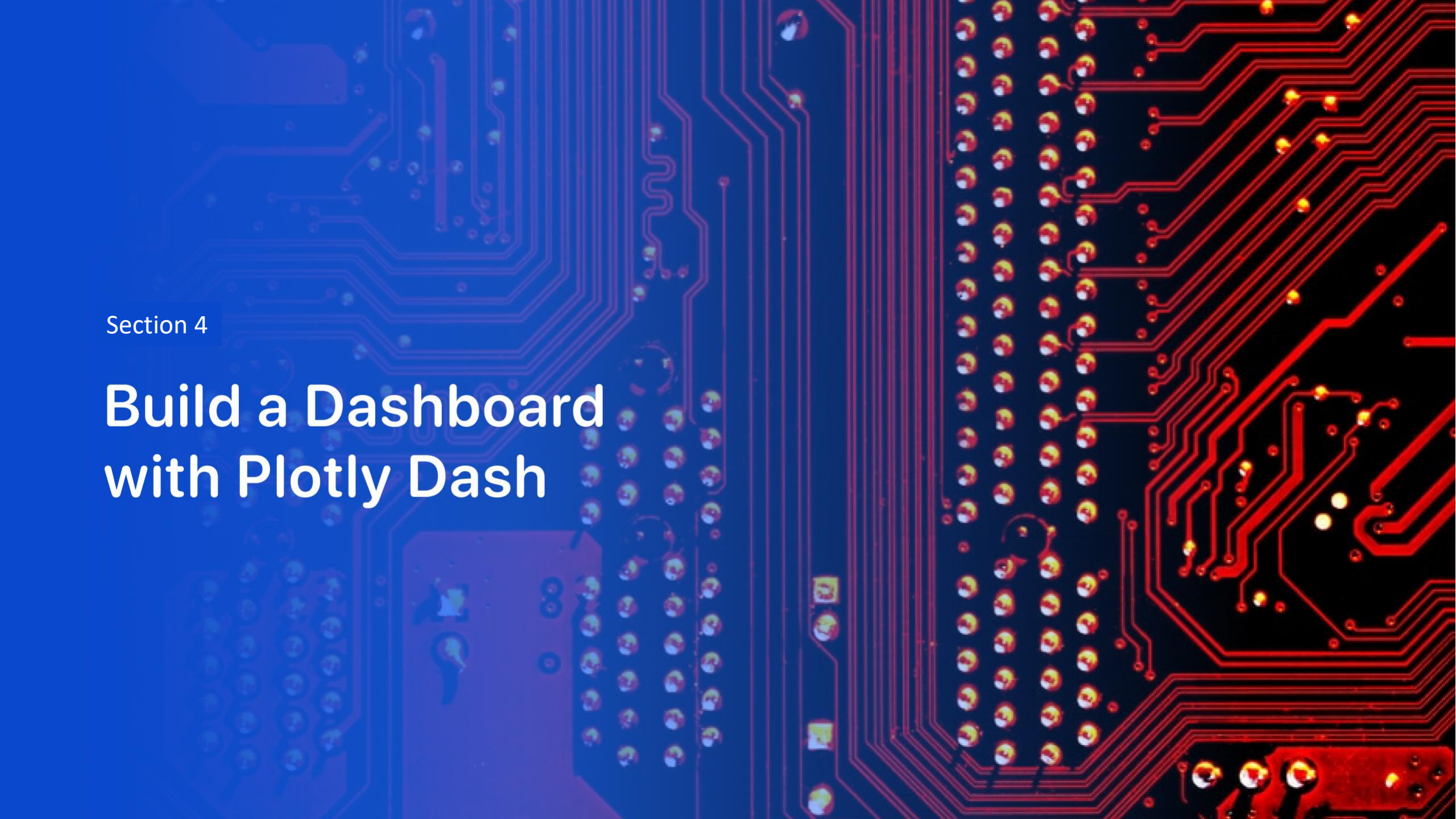
- **Green markers** indicate **successful landings**.
- **Red markers** indicate **failed landings**.



Distance b/w Launch Site & Proximities

- Launch site proximity to **highways and coastline** measured.
- Close distance to **coastline** supports safe rocket recovery



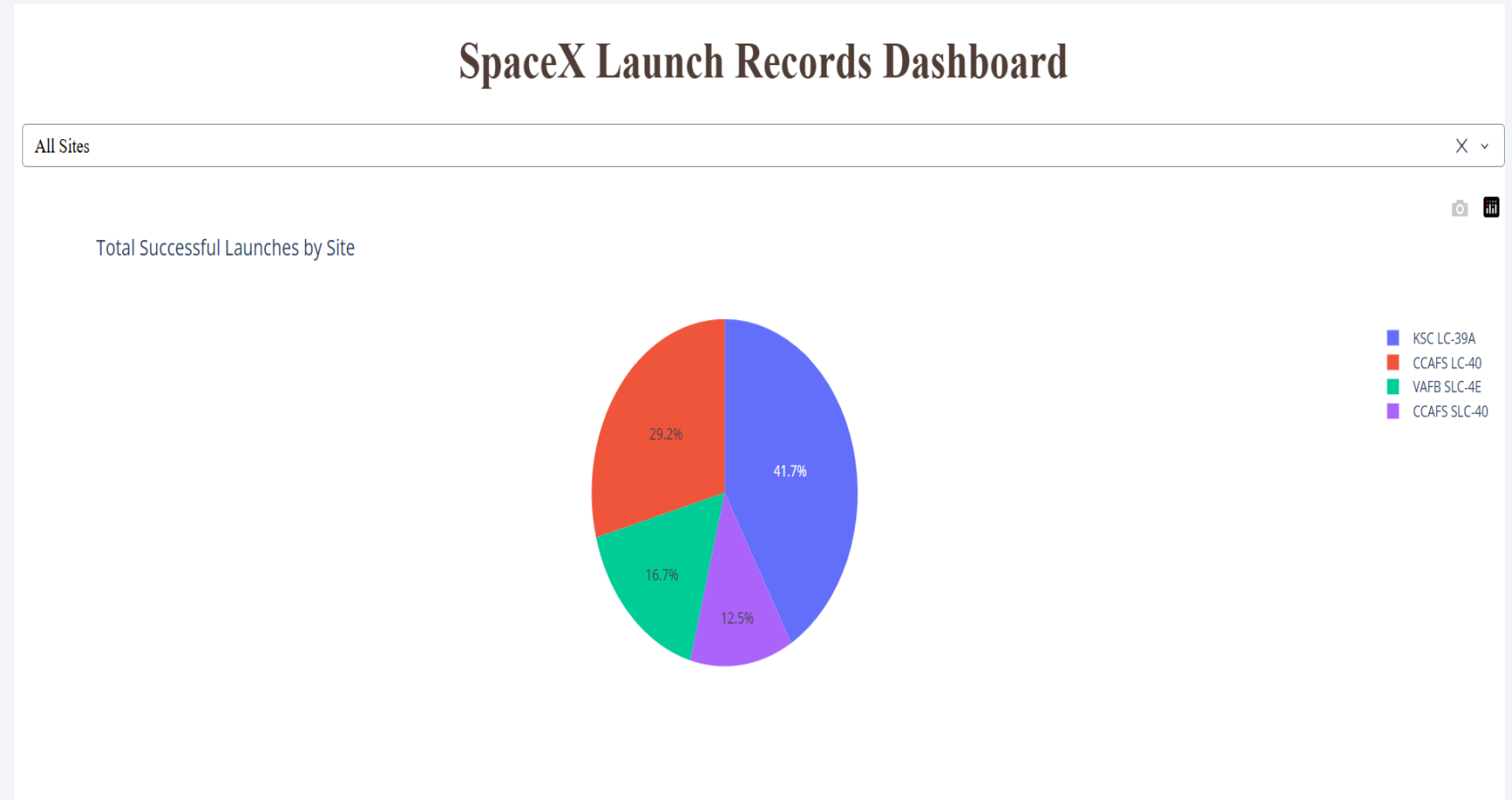


Section 4

Build a Dashboard with Plotly Dash

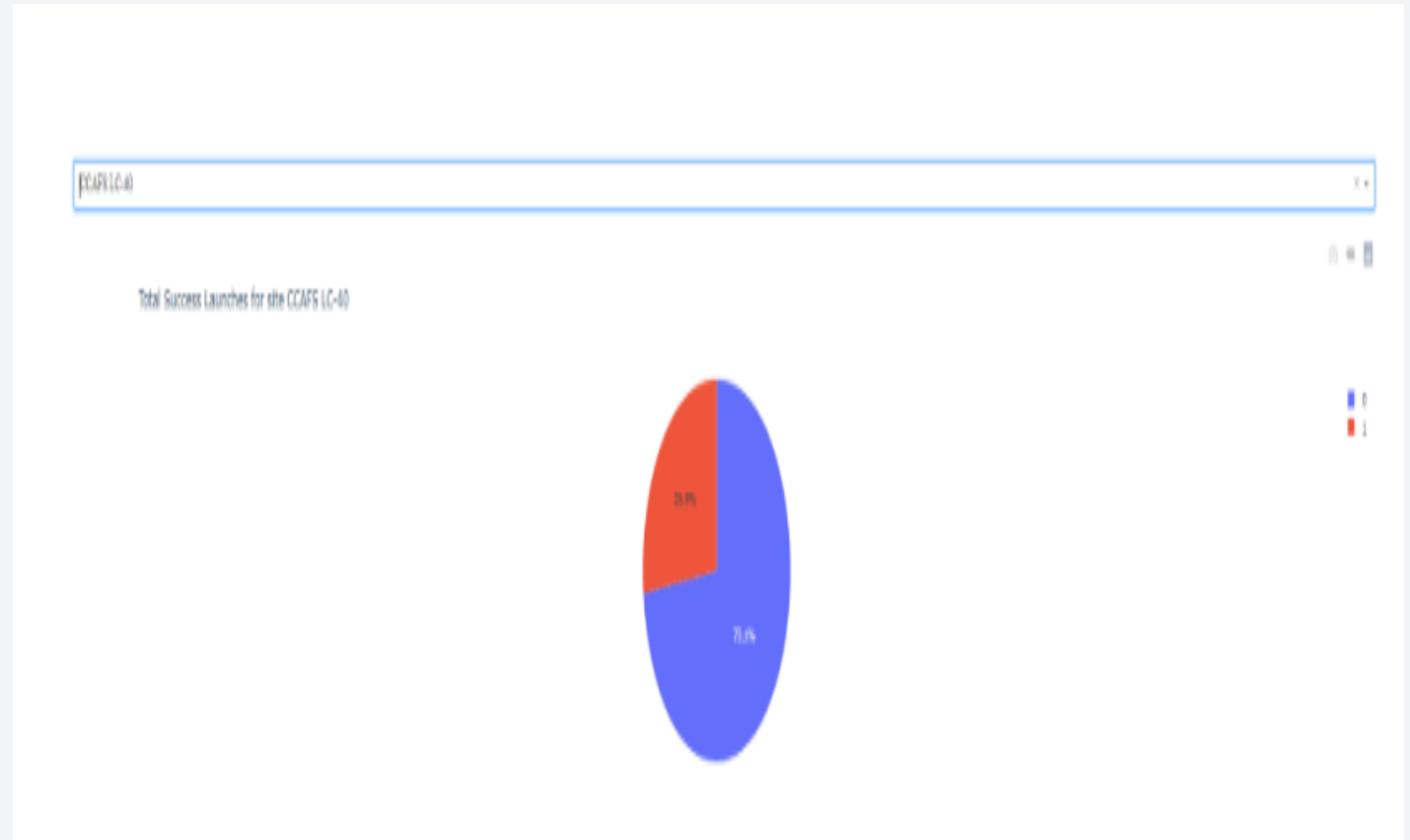
Success Rate for All Locations

- Pie chart compares successful launches.
- KSC achieved highest number of successful missions.



Success Rate for Specific Location

- Site filter enables comparison of launch locations.



Success-Payload Scatter Plot



- Payload slider reveals higher payloads generally have better landing success.



Section 5

Predictive Analysis (Classification)

Classification Accuracy

Model	Accuracy
Logistic Regression	94%
Decision Tree	88%
SVM	83%
KNN	83%

- **Logistic Regression** achieved **the highest prediction accuracy** and was selected as the final model.

Confusion Matrix

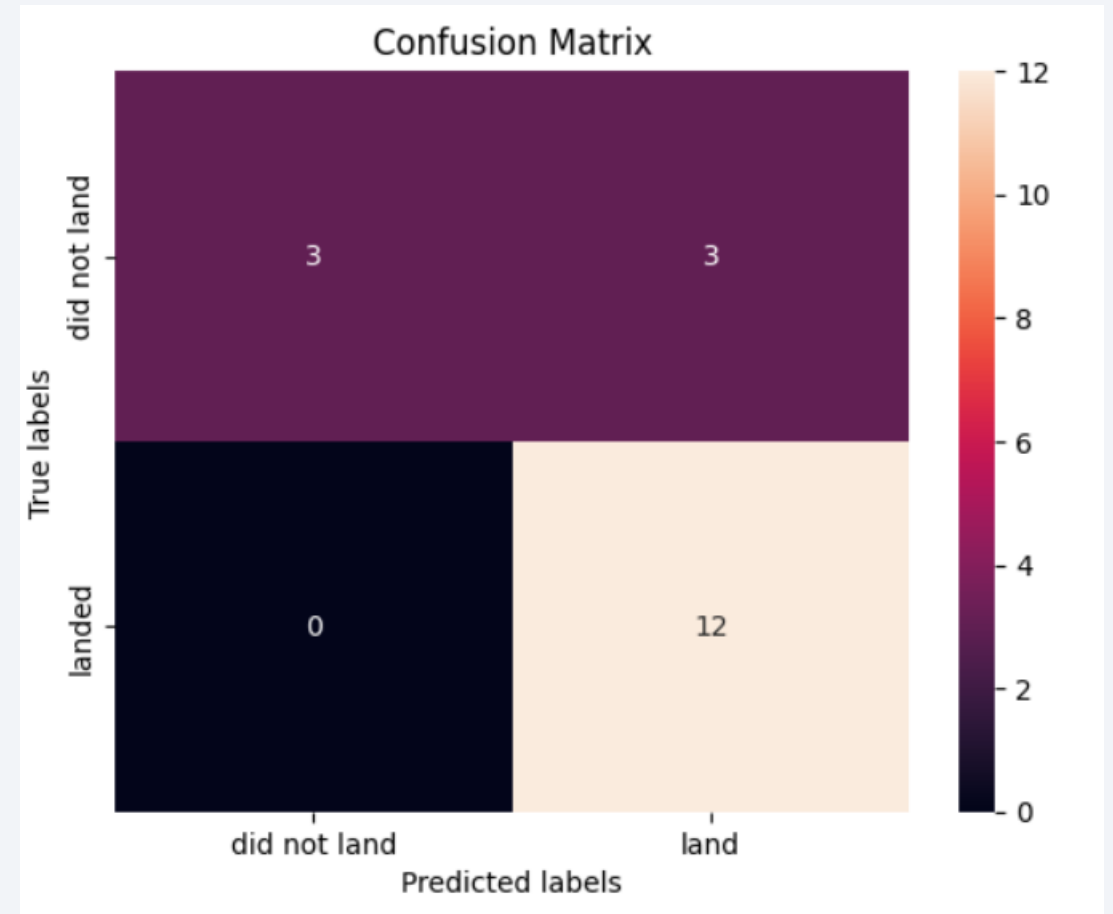
The Confusion Matrix of Logistic Regression

True Positive: 12

False Positive : 3

True Negative: 3

False Negative: 0



Conclusions

- Successfully collected launch data from SpaceX API and Wikipedia.
- Cleaned and prepared the dataset for analysis.
- Performed exploratory data analysis using visualization and SQL.
- Built interactive maps and dashboards for better insights.
- Developed multiple machine learning models to predict Falcon 9 landing success.
- KNN (or your best model) achieved the highest prediction accuracy.

Appendix

Data Collection

We will import the following libraries.

```
[2]: # Pandas is a software library written for the Python programming language for data manipulation and analysis.
import pandas as pd
# NumPy is a library for the Python programming language, adding support for large, multi-dimensional arrays and matrices, along with a large collection of mathematical functions.
import numpy as np
```

Data Analysis

Load Space X dataset, from last section.

```
[3]: df=pd.read_csv("https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM-DS03210-SkillNetwork/datasets/dataset_part_1.csv")
df.head(10)
```

```
[3]:
```

	FlightNumber	Date	BoosterVersion	PayloadMass	Orbit	LaunchSite	Outcome	Flights	GridFins	Reused	Legs	LandingPad	Block	ReusedCount	Se
0	1	2010-06-04	Falcon 9	6104.959412	LEO	CCAFS SLC 40	None	1	False	False	False	NaN	1.0	0	BOX
1	2	2012-05-22	Falcon 9	525.000000	LEO	CCAFS SLC 40	None	1	False	False	False	NaN	1.0	0	BOX
2	3	2013-03-01	Falcon 9	677.000000	ISS	CCAFS SLC 40	None	1	False	False	False	NaN	1.0	0	BOX
3	4	2013-09-29	Falcon 9	500.000000	PO	VAFB SLC 4E	False Ocean	1	False	False	False	NaN	1.0	0	BTI
4	5	2013-12-03	Falcon 9	3170.000000	GTO	CCAFS SLC 40	None	1	False	False	False	NaN	1.0	0	BTI
5	6	2014-01-06	Falcon 9	3325.000000	GTO	CCAFS SLC 40	None	1	False	False	False	NaN	1.0	0	BTI

```
[ ]: def date_time(table_cells):
    """
    This function returns the date and time from the HTML table cell
    Input: the element of a table data cell extracts extra row
    """
    return [data_time.strip() for data_time in list(table_cells.strings)][0:2]

def booster_version(table_cells):
    """
    This function returns the booster version from the HTML table cell
    Input: the element of a table data cell extracts extra row
    """
    out=''.join([booster_version for i,booster_version in enumerate(table_cells.strings) if i%2==0][0:-1])
    return out

def landing_status(table_cells):
    """
    This function returns the landing status from the HTML table cell
    Input: the element of a table data cell extracts extra row
    """
    out=[_for i in table_cells.strings][0]
    return out

def get_mass(table_cells):
    mass=suppl.loads(table_cells).find('mass')
    if mass:
        mass.find("kg")
        new_mass=mass.find("kg").text
    else:
        new_mass=0
    return new_mass

def extract_column_from_header(row):
    """
```

EDA SQL Queries

Task 1

Display the names of the unique launch sites in the space mission

```
[ ]: %sql Select Distinct Launch_Site From SPACEXTBL;
```

Task 2

Display 5 records where launch sites begin with the string 'CCA'

```
[ ]: %sql Select * From SPACEXTBL WHERE Launch_Site Like "CCA%";
```

Task 3

Display the total payload mass carried by boosters launched by NASA (CRS)

```
[ ]: %sql SELECT BOOSTER_VERSION, SUM(PAYLOAD_MASS__KG_) FROM SPACEXTBL GROUP BY BOOSTER_VERSION;
```

Task 4

Display average payload mass carried by booster version F9 v1.1

```
[ ]: %sql SELECT BOOSTER_VERSION, AVG(PAYLOAD_MASS__KG_) FROM SPACEXTBL WHERE BOOSTER_VERSION = 'F9 v1.1' GROUP BY BOOSTER_VERSION;
```

Task 5

List the date when the first successful landing outcome in ground pad was achieved.

Hint: Use min function

```
[ ]: %sql SELECT * FROM SPACEXTBL WHERE LANDING_OUTCOME = 'Success (ground pad)' Order by Date Asc Limit 1;
```

List the names of the boosters which have success in drone ship and have payload mass greater than 4000 (in kg)

```
[ ]: %sql SELECT BOOSTER_VERSION FROM SPACEXTBL WHERE LANDING_OUTCOME = 'Success (drone ship)' AND (PAYLOAD_MASS__KG_ > 4000 AND PAYLOAD_MASS__KG_ < 40000);
```

Task 7

List the total number of successful and failure mission outcomes

```
[ ]: %sql SELECT MISSION_OUTCOME, COUNT(*) FROM SPACEXTBL GROUP BY MISSION_OUTCOME;
```

Task 8

List all the booster versions that have carried the maximum payload mass, using a subquery with a suitable aggregate function.

```
[ ]: %sql SELECT BOOSTER_VERSION FROM SPACEXTBL WHERE PAYLOAD_MASS__KG_ = (SELECT MAX(PAYLOAD_MASS__KG_) FROM SPACEXTBL);
```

Task 9

List the records which will display the month names, failure landing, outcomes in drone ship, booster versions, launch site for the months in year 2015.

Note: SQLite does not support monthnames. So you need to use substr(Date, 6,2) as month to get the months and substr(Date,0,5)='2015' for year.

```
[ ]: %sql SELECT SUBSTR(Date,6,2) AS MONTH, LANDING_OUTCOME, BOOSTER_VERSION, LAUNCH_SITE FROM SPACEXTBL WHERE SUBSTR(Date,0,5) = '2015' AND LANDING_OUTCOME = 'Failure (drone ship)';
```

Task 10

Rank the count of landing outcomes (such as Failure (drone ship) or Success (ground pad)) between the date 2010-06-04 and 2017-03-20, in descending order.

```
[ ]: %sql SELECT LANDING_OUTCOME,COUNT(*) AS TOTAL_OUTCOME,RANK() OVER(ORDER BY Count(*) DESC) AS RANKING FROM SPACEXTBL WHERE DATE BETWEEN '2010-06-04' AND '2017-03-20';
```

Classification Models

TASK 4

Create a logistic regression object then create a GridSearchCV object logreg_cv with cv = 10. Fit the object to find the best parameters from the dictionary parameters.

```
16: parameters = {'C':[0.01,0.1,1],
               'penalty':['l2'],
               'solver':['lbfgs']}
17: parameters = {'C':[0.01,0.1,1],'penalty':['l2'],'solver':['lbfgs']} # L1 Lasso L2 ridge
logreg = LogisticRegression()
18: logreg_cv = GridSearchCV(
    estimator=logreg,
    param_grid=parameters,
    cv=10
)
logreg_cv.fit(X, Y)
```

We output the GridSearchCV object for logistic regression. We display the best parameters using the data attribute best_params_ and the accuracy on the validation data using the data attribute best_score_.

```
19: print(logreg_cv.best_params_)
print(logreg_cv.best_score_)
```

TASK 6

Create a support vector machine object then create a GridSearchCV object svm_cv with cv = 10. Fit the object to find the best parameters from the dictionary parameters.

```
5: parameters = {'kernel':('linear', 'rbf','poly','rbf', 'sigmoid'),
               'C': np.logspace(-3, 3, 5),
               'gamma':np.logspace(-3, 3, 5)}
svm = SVC()
6: svm_cv = GridSearchCV(
    estimator=svm,
    param_grid=parameters,
    cv=10
)
svm_cv.fit(X_train, Y_train)
7: print(svm_cv.best_params_)
print(svm_cv.best_score_)
```

tuned hyperparameters : (best parameters) {'C': 1.0, 'gamma': 0.03162277660168379, 'kernel': 'sigmoid'}
accuracy : 0.8482142857142856

Thank you!

